UC
UNIVERSIDAD DE CANTABRIA

Tecnologías de Desarrollo de Aplicaciones Empresariales sobre Internet

Máster en Ingeniería Informática

Documento de Especificación API REST

Polaflix - Sistema de Streaming Simplificado

Autor: Miguel Martín
Fecha: 19 de abril de 2026

Índice

1. Introducción **2**

2. Modelos de Respuesta (DTOs) **2**

3. Recurso: Usuarios **2**
3.1. Crear un nuevo usuario 2
3.2. Obtener detalle de usuario por ID 2
3.3. Actualizar marcador de lectura de un usuario 3

4. Recurso: Series **3**
4.1. Crear una nueva Serie 3
4.2. Obtener detalle de una Serie 3
4.3. Obtener detalle de un Capítulo 4

5. Recurso: Facturas **5**
5.1. Obtener detalle de una Factura 5
5.2. Obtener desglose de visualizaciones de una Factura 7

6. Códigos de Estado HTTP Utilizados **7**

1. Introducción

Este documento detalla la interfaz de programación de aplicaciones (API) RESTful para el sistema Polaflix. Todas las respuestas están formateadas en JSON y hacen uso intensivo de Data Transfer Objects (DTOs) para optimizar la carga de red, evitar referencias circulares y ofrecer un balance adecuado de información al cliente sin requerir excesivas peticiones adicionales.

URL Base del entorno local: `http://localhost:8080`

2. Modelos de Respuesta (DTOs)

Las entidades de dominio se mapean a los siguientes DTOs. Se hace uso de DTOs de Resumen.ªnidados para proporcionar información relacional útil (como el nombre de la serie o usuario asociados) sin sobrecargar el *payload*:

- **UsuarioDTO:** `{idUsuario, nombreUsuario, tipo, seriesPendientesIds, seriesTerminadasIds, seriesEmpezadasIds, facturasIds}`
- **UsuarioResumenDTO:** `{idUsuario, nombreUsuario, tipo}`
- **SerieDTO:** `{idSerie, nombreSerie, descripcion, categoria, actores, creadores, temporadas}` *(actores y creadores ahora exponen la información completa de las personas asociadas).*
- **SerieResumenDTO:** `{idSerie, nombreSerie, categoria, temporadasIds}` *(Evita cargar los actores o el detalle de temporadas en listados anidados, pero incluye la categoría para contexto de facturación).*
- **FacturaDTO:** `{idFactura, mes, total, usuario, visualizaciones}` *(usuario es un UsuarioResumenDTO y visualizaciones es un `Set<VisualizacionDTO>`).*
- **VisualizacionDTO:** `{idVisualizacion, fechaVisualizacion, capitulo, numTemporada, precioCobrado, serie}` *(serie es un SerieResumenDTO y capitulo es un CapituloDTO).*
- **TemporadaDTO:** `{numeroTemporada, capitulosTitulos}`
- **CapituloDTO:** `{titulo, descripcion, enlace}`
- **RegistroSerieUsuarioDTO:** `{id, ultimoCapituloNumTemporada, ultimoCapituloNumCapitulo, usuario, serie}` *(usuario es UsuarioResumenDTO y serie es SerieResumenDTO).*

3. Recurso: Usuarios

Ruta base: `/usuarios`

3.1. Crear un nuevo usuario

- **Endpoint:** `POST /usuarios`
- **Cuerpo de Petición:** Entidad `Usuario` en formato JSON.
- **Respuesta Exitosa:** `201 Created` - Devuelve el `UsuarioDTO` recién creado.

3.2. Obtener detalle de usuario por ID

- **Endpoint:** `GET /usuarios/{id}`
- **Respuesta Exitosa:** `200 OK`

- **Ejemplo de Payload (Usuario con pago por visualización):**

```
{
  "idUsuario": 2,
  "nombreUsuario": "juan",
  "tipo": "PAGOPORVISUALIZACION",
  "seriesPendientesIds": [],
  "seriesTerminadasIds": [],
  "seriesEmpezadasIds": [2, 3],
  "facturasIds": [2]
}
```

3.3. Actualizar marcador de lectura de un usuario

- **Endpoint:** `PUT /usuarios/{usuarioId}/registros/{serieId}/temporadas/{numTemp}/capitulos/{n`
- **Parámetros:** Pasados íntegramente por *path* para identificar unívocamente el recurso exacto en la jerarquía.
- **Respuesta Exitosa:** `200 OK` - Devuelve `RegistroSerieUsuarioDTO`.
- **Ejemplo de Payload:**

```
{
  "id": 2,
  "ultimoCapituloNumTemporada": 1,
  "ultimoCapituloNumCapitulo": 1,
  "usuario": {
    "idUsuario": 2,
    "nombreUsuario": "juan",
    "tipo": "PAGOPORVISUALIZACION"
  },
  "serie": {
    "idSerie": 2,
    "nombreSerie": "C digo  H brido",
    "categoria": "SILVER",
    "temporadasIds": [1]
  }
}
```

4. Recurso: Series

Ruta base: `/series`

4.1. Crear una nueva Serie

- **Endpoint:** `POST /series`
- **Respuesta Exitosa:** `201 Created` - Devuelve `SerieDTO`.

4.2. Obtener detalle de una Serie

- **Endpoint:** `GET /series/{id}`

- **Respuesta Exitosa:** `200 OK`
- **Ejemplo de Payload (Serie Silver):**

```
{
  "idSerie": 2,
  "nombreSerie": "C digo H brido",
  "descripcion": "Una trama de hacking y conspiraci n.",
  "categoria": "SILVER",
  "actores": [
    {
      "id": 3,
      "nombre": "Laura",
      "primerApellido": "Gomez",
      "segundoApellido": "P rez"
    }
  ],
  "creadores": [
    {
      "id": 4,
      "nombre": "Diego",
      "primerApellido": "Ruiz",
      "segundoApellido": "Ortiz"
    }
  ],
  "temporadas": {
    "1": {
      "numeroTemporada": 1,
      "capitulosTitulos": {
        "1": "C digo en la sombra"
      }
    }
  }
}
```

4.3. Obtener detalle de un Capítulo

- **Endpoint:** `GET /series/{serieId}/temporadas/{numTemp}/capitulos/{numCap}`
- **Respuesta Exitosa:** `200 OK`
- **Ejemplo de Payload:**

```
{
  "titulo": "Nacimiento de una estrella",
  "descripcion": "C mo se forman las estrellas en la nebulosa.",
  "enlace": "http://example.com/video/1"
}
```

5. Recurso: Facturas

Ruta base: `/facturas`

5.1. Obtener detalle de una Factura

- **Endpoint:** `GET /facturas/{id}`
- **Descripción:** Muestra el total autocalculado de la factura. Destaca el balance de información anidando el resumen del usuario y todas sus visualizaciones con sus respectivos datos de la serie y el capítulo asociado.
- **Ejemplo de Payload (Usuario PAGOPORVISUALIZACION):**

```
{
  "idFactura": 2,
  "mes": "2026-03",
  "total": 2.25,
  "usuario": {
    "idUsuario": 2,
    "nombreUsuario": "juan",
    "tipo": "PAGOPORVISUALIZACION"
  },
  "visualizaciones": [
    {
      "idVisualizacion": 2,
      "fechaVisualizacion": "2026-04-14T10:00:00.000+00:00",
      "capitulo": {
        "titulo": "C digo en la sombra",
        "descripcion": "Un thriller tecnol gico con giros
   inesperados.",
        "enlace": "http://example.com/video/2"
      },
      "numTemporada": 1,
      "precioCobrado": 0.75,
      "serie": {
        "idSerie": 2,
        "nombreSerie": "C digo H brido",
        "categoria": "SILVER",
        "temporadasIds": [1]
      }
    },
    {
      "idVisualizacion": 3,
      "fechaVisualizacion": "2026-04-14T10:05:00.000+00:00",
      "capitulo": {
        "titulo": "Ascenso al dominio",
        "descripcion": "Una serie  pica  de fantas a con batallas
   y magia.",
        "enlace": "http://example.com/video/3"
      },
      "numTemporada": 1,
      "precioCobrado": 1.50,
      "serie": {
        "idSerie": 3,
        "nombreSerie": "Tronos de Ceniza",
        "categoria": "GOLD",
        "temporadasIds": [1]
      }
    }
  ]
}
```

5.2. Obtener desglose de visualizaciones de una Factura

- **Endpoint:** `GET /facturas/{id}/visualizaciones`
- **Respuesta Exitosa:** `200 OK`
- **Ejemplo de Payload:** Devuelve el array interno del campo `"visualizaciones"` mostrado en la petición anterior.

6. Códigos de Estado HTTP Utilizados

La API utiliza los códigos de estado estándar de HTTP para indicar el éxito o fallo de las peticiones, cubriendo las posibles situaciones durante la ejecución de las peticiones:

- **200 OK:** La petición se ha procesado correctamente y se devuelve el recurso.
- **201 Created:** El recurso ha sido creado exitosamente (ej. POST a `/usuarios`).
- **204 No Content:** Operación exitosa que no requiere devolver cuerpo (ej. `DELETE`).
- **400 Bad Request:** Parámetros o cuerpo de la petición mal formados.
- **404 Not Found:** El recurso solicitado no existe en la base de datos o en la ruta especificada.